Python Code

```
"""
Code to calculate the circle that passes through three given points.

Fill in each function with your code (including fixing the return
statement).
"""

# imports
import math
import comp140_module1 as circles

def distance(point0x, point0y, point1x, point1y):
    """
    Computes the distance between two points.

    inputs:
        -point0x: a float representing the x-coordinate of the first point
        -point0y: a float representing the y-coordinate of the first point
        -point1x: a float representing the x-coordinate of the second point
        -point1y: a float representing the y-coordinate of the second point

    returns: a float that is the distance between the two points
    """
    #distance formula
    return math.sqrt((point0x - point1x) ** 2 + (point1y - point0y) ** 2)

def midpoint(point0x, point0y, point1x, point1y):
    """
    Computes the midpoint between two points.

    inputs:
        -point0x: a float representing the x-coordinate of the first point
        -point0y: a float representing the y-coordinate of the first point
        -point1x: a float representing the x-coordinate of the second point
        -point1y: a float representing the y-coordinate of the second point

    returns: two floats that are the x- and y-coordinates of the midpoint
    """
    # midpoint formula
    return (point0x + point1x)/2, (point0y + point1y)/2

def slope(point0x, point0y, point1x, point1y):
    """
    Computes the slope of the line that connects two given points.

    The x-values of the two points, point0x and poin1x, must be different.

    inputs:
        -point0x: a float representing the x-coordinate of the first point.
        -point0y: a float representing the y-coordinate of the first point
```

```
        -point1x: a float representing the x-coordinate of the second point.
        -point1y: a float representing the y-coordinate of the second point

    returns: a float that is the slope between the points
    """
    # slope formula
    return (point1y - point0y)/(point1x - point0x)

def perp(lineslope):
    """
    Computes the slope of a line perpendicular to a given slope.

    input:
        -lineslope: a float representing the slope of a line.
                    Must be non-zero

    returns: a float that is the perpendicular slope
    """

    return -1/lineslope


def intersect(slope0, point0x, point0y, slope1, point1x, point1y):
    """
    Computes the intersection point of two lines.

    The two slopes, slope0 and slope1, must be different.

    inputs:
        -slope0: a float representing the slope of the first line.
        -point0x: a float representing the x-coordinate of the first point
        -point0y: a float representing the y-coordinate of the first point
        -slope1: a float representing the slope of the second line.
        -point1x: a float representing the x-coordinate of the second point
        -point1y: a float representing the y-coordinate of the second point

    returns: two floats that are the x- and y-coordinates of the intersection
    point
    """
    #y = m1 * x - m1 * x1) + y1
    #y = m2 * x - m2 * x2) + y2
    # x_intersect = - m1 * x1 + y1 - y2 + m2 * x2 / (m2 - m1)
    # y_intersect = y = m2 * x - m2 * x2) + y2
    # point slope form to find coords of intersection

    x_intersect = (point0y - slope0 * point0x - point1y + slope1 * point1x) / (slope1
- slope0)
    y_intersect = slope1 * x_intersect - slope1 * point1x + point1y

    return x_intersect, y_intersect

def make_circle(point0x, point0y, point1x, point1y, point2x, point2y):
    """
    Computes the center and radius of a circle that passes through
```

```
    thre given points.

    The points must not be co-linear and no two points can have the
    same x or y values.

    inputs:
        -point0x: a float representing the x-coordinate of the first point
        -point0y: a float representing the y-coordinate of the first point
        -point1x: a float representing the x-coordinate of the second point
        -point1y: a float representing the y-coordinate of the second point
        -point2x: a float representing the x-coordinate of the third point
        -point2y: a float representing the y-coordinate of the third point

    returns: three floats that are the x- and y-coordinates of the center
    and the radius
    """
    #Connect a pair of points with a line.
    #Find the midpoint of that line.
    #Draw a new line that is perpendicular to the original line and passes through the
midpoint.
    #Repeat steps 1 through 3 for another pair of points.
    # The point at which the two perpendicular lines intersect is the center of the
circle.
    # The distance from the center to any of the three original points is the radius

    midpoint1_x, midpoint1_y = midpoint(point0x, point0y, point1x, point1y)
    midpoint2_x, midpoint2_y = midpoint(point1x, point1y, point2x, point2y)
    perp_1 = perp(slope(point0x, point0y, point1x, point1y))
    perp_2 = perp(slope(point1x, point1y, point2x, point2y))
    intersect_x, intersect_y = intersect(perp_1, midpoint1_x,
                                midpoint1_y, perp_2, midpoint2_x, midpoint2_y)
    radius = distance(intersect_x, intersect_y, point0x, point0y)

    return intersect_x, intersect_y, radius

# Run GUI - uncomment the line below after you have
#           implemented make_circle
circles.start(make_circle)

```